

Cosmo OS – Beginner Guide

A complete introduction to the cosmo-os project: what it is, how it is built, and how data flows through the system. Written in simple English.

Generated from the cosmo-os repository.

1. What is this project?

Cosmo OS is a business operations platform for an e-commerce company (Cosmetics.lk and related brands). It helps staff:

- Receive and manage Shopify orders
- Sync orders to ERPNext (accounting/ERP system)
- Handle fulfillment (pack, dispatch, delivery)
- Manage staff, roles, and permissions
- Track contacts, complaints, returns, exchanges
- Run reports and dashboards
- Support riders via a mobile app (Cosmo Rider)

The same codebase also runs Vault OS (supplement business) – one app, different databases.

Simple mental model: Shopify (online shop) ↔ Cosmo OS (control center) ↔ ERPNext (finance) ↔ Riders (delivery).

2. SDLC – How the team builds software

SDLC = Software Development Life Cycle (the steps from idea to production). This project uses a modern, agile-style workflow:

Phase	What happens
Planning	Features added as needed (orders, fulfillment, contacts, etc.)
Development	Developers write code locally with <code>npm run dev</code>
Version control	Git + GitHub – code in branches, merged via Pull Requests
Database changes	Edit <code>prisma/schema.prisma</code> ↔ create migration ↔ deploy to all 3 DBs
Testing	Vitest unit tests + TypeScript checks (especially mobile)
CI	GitHub Actions runs tests on every PR
Deployment	Vercel (Next.js) + Neon (PostgreSQL) + EAS (mobile builds)

Daily developer flow:

```
git pull & npm install & npm run db:deploy & npm run dev & code & test & PR & merge & deploy
```

3. Tech stack

Web app (main product)

Layer	Technology	Simple meaning
Framework	Next.js 16 (App Router)	React framework — pages + API in one project
UI	React 19 + Tailwind CSS 4	User interface + styling
Components	shadcn/ui + Radix UI	Pre-built accessible UI pieces
Language	TypeScript	JavaScript with types — fewer bugs
Database	PostgreSQL (Neon cloud)	Stores all business data
ORM	Prisma	Talks to the database using TypeScript
Auth	Auth0	Login/logout — you don't build passwords yourself
Validation	Zod	Checks all input is safe and correct
Charts	Recharts	Dashboard graphs
Files	Cloudinary, Vercel Blob	Image/file storage

Mobile app

Layer	Technology
Framework	Expo + React Native
Routing	Expo Router
Backend	Same Next.js API (/api/mobile/v1/*)

External systems

System	Role
Shopify	Online store — sends order webhooks
ERPNext	Invoices, customers, accounting
Auth0	User login for web + mobile
SMS (Hutch)	Order notifications
Maileroo	Email

4. Protocols & communication

Protocol = the language two systems use to talk.

Protocol	Where used
HTTPS	All web and API traffic (encrypted)

Protocol	Where used
HTTP REST	Frontend calls /api/admin/... with GET/POST/PATCH/DELETE
JSON	Data format for APIs and webhooks
OAuth / OIDC	Auth0 login flow (/auth/login, session cookies)
Bearer token	Mobile app sends Authorization: Bearer <token>
Webhooks	Shopify and ERPNext push events to /api/webhooks/...
HMAC-SHA256	Shopify signs webhooks — app verifies they are real
PostgreSQL wire protocol	Prisma ↔ Neon database

Important rule: The browser is NOT trusted. Every API route checks auth and validates input on the server.

5. Architecture (big picture)

This is a monorepo with a layered, full-stack Next.js design.

Architecture style

- Full-stack monolith — one Next.js app (not microservices)
- Server-first — many pages fetch data on the server before showing UI
- API routes as backend — no separate Express server
- Shared lib/ — business logic reused by pages and APIs
- Multi-tenant by company — each Company has its own data (companyId on rows)
- RBAC — Role-Based Access Control (permissions like orders.read)

Clients (Web, Mobile, Shopify, ERPNext)

↔

Next.js: app/ (pages) + app/api/ (REST) + lib/ (logic)

↔

PostgreSQL via Prisma

↔

External: ERPNext, SMS, Email

6. Folder structure

cosmo-os/

```

├── app/           Next.js App Router (routes = folders)
│   ├── layout.tsx  Root HTML shell, theme, Auth0
│   ├── (dashboard)/ Protected admin area
│   │   ├── layout.tsx Checks login, wraps sidebar
│   │   └── dashboard/ Each page = one URL
│   └── api/        Backend endpoints
│       ├── admin/  Staff dashboard APIs
│       ├── mobile/v1/ Rider app APIs
│       ├── webhooks/ Shopify, ERPNext
│       └── cron/    Scheduled jobs

```

- components/ UI (ui molecules organisms templates)
- lib/ Business logic (the brain)
- prisma/ Database schema + migrations
- mobile/rider-app/ Separate Expo project
- scripts/ One-off tools
- .github/workflows/ CI pipeline

Next.js routing: folder name = URL path. Example: app/(dashboard)/dashboard/orders/page.tsx → dashboard/orders

(dashboard) is a route group — organizes files but does NOT appear in the URL.

7. How logic flows

A. User opens the dashboard

- Browser → proxy.ts → Auth0 (check session)
- dashboard/layout.tsx → lib/rbac (user + permissions)
- page.tsx → HTML sent to browser

Key file: proxy.ts protects routes and lets webhooks through without login.

B. User clicks Save on a form

- React Component → fetch POST /api/admin/...
- requirePermission() — auth check
- Zod validation — input check
- lib/*.ts — business work
- prisma → PostgreSQL
- JSON response → notify.success()

Every API follows: 1) Auth 2) Validate 3) lib/ logic 4) JSON response

C. Shopify sends a new order

- Shopify → POST /api/webhooks/shopify/orders
- Verify HMAC signature
- Validate with Zod
- processOrderWebhook
- Save Order, line items, sync ERPNext, send SMS, assign merchant
- On failure: FailedOrderWebhook + auto-retry

D. Page-data pattern (performance)

Instead of 5 API calls, one endpoint returns everything:

/dashboard/products → GET /api/admin/product-items/page-data

Logic lives in lib/page-data/product-items.ts, called by the API route.

8. Database model (simple view)

Company

- Users (staff, Roles & Permissions)
- CompanyLocations (Shopify stores / warehouses)
- Orders (from Shopify)
- ProductItems (catalog)
- Contacts (CRM)
- Customers
- ErpNextInstance (ERP connection settings)

Prisma defines models in `prisma/schema.prisma`. Migrations are SQL files that change the live database.

Three environments: `.env.vault` (Vault OS), `.env.cosmo-dev` (dev), `.env.cosmo-prod` (production). Switch with: `npm run env:use cosmo-dev`

9. Security & permissions (RBAC)

RBAC = each user has roles; each role has permissions. Examples:

- `orders.read` — view orders
- `orders.manage` — dispatch, fulfillment
- `staff.manage` — edit employee profiles
- `settings.company` — company settings

Defined in `lib/rbac.ts`. APIs call `requirePermission()` before doing work.

Golden rule: Never trust the browser. Validate everything server-side with Zod (`lib/validation.ts`).

10. Best practices in this project

Practice	What it means here
TypeScript everywhere	Catch mistakes before runtime
Server-side validation (Zod)	<code>lib/validation.ts</code> + domain schemas
Single page-data APIs	Fewer round trips, faster pages
Server prefetch	Dashboard loads data on server, passes to client
server-only imports	Secret logic never ships to browser
Atomic design for UI	<code>ui</code> → <code>molecules</code> → <code>organisms</code> → <code>templates</code>
Audit logs	Track who changed what
Performance logging	<code>createPerfLogger()</code> in hot paths
Loading UX	Spinners, disable buttons, toast notifications
Git + CI	Tests run automatically on PRs
Migration discipline	Never <code>db:push</code> on shared/prod DBs

11. UI component layers

`ui/` • Button, Input, Card (smallest)

molecules/ Login form, stat card (small groups)
organisms/ Orders panel, staff panel (big sections)
templates/ Dashboard shell (full page frame)

Pages in app/ stay thin — they mostly import an organism and pass data.

12. Mobile app (Cosmo Rider)

Separate Expo project in mobile/rider-app/. Calls the same backend at /api/mobile/v1/*. Routes are thin; logic is in src/hooks/ and src/api/. One app supports multiple companies (Cosmetics + Vault) with separate tokens per tenant.

13. How to learn this project (beginner path)

Week 1 — Setup & tour

- Read README.md
- Run: `npm install` `npm run env:use cosmo-dev` `npm run dev`
- Open `http://localhost:3000` and explore
- Skim `prisma/schema.prisma` — see Company, User, Order

Week 2 — Follow one feature (Orders)

- Page: `app/(dashboard)/dashboard/orders/...`
- Panel: `components/organisms/...`
- API: `app/api/admin/orders/...`
- Logic: `lib/order-webhook-process.ts`, `lib/erpnext-sync.ts`
- DB: Order model in Prisma

Week 3 — Auth & permissions

- `proxy.ts`
- `lib/auth0.ts`
- `lib/rbac.ts`
- `app/(dashboard)/layout.tsx`

Week 4 — Integrations

- `app/api/webhooks/shopify/orders/route.ts`
- `lib/shopify-webhook.ts`
- `lib/erpnext-sync.ts`

Week 5 — Patterns to copy

- Any page-data route + lib/page-data/ file
- .cursor/rules/security-validation.mdc
- .cursor/rules/performance-optimization.mdc

14. Useful commands

```
npm run dev      # Start web app
npm test        # Run tests
npm run lint     # Check code style
npm run db:generate # After schema changes
npm run db:deploy:cosmo-dev # Apply migrations
npm run env:use cosmo-dev # Switch environment
```

15. Mental model (remember this)

```
URL (app/)  ↳ Page shows UI (components/)
            ↳ fetch
API (app/api/)  ↳ Auth (rbac)  ↳ Validate (Zod)
                ↳
Business logic (lib/)  ↳ Database (Prisma/PostgreSQL)
                        ↳
External systems (Shopify, ERPNext, SMS, Email)
```

Cosmo OS is the middle layer that connects the online shop, warehouse team, finance, and delivery riders — with strict login, permissions, and validated data at every step.